BUILD GUIDE

GigScore: an alternative-data credit decision engine for credit-invisible gig workers, built on the L&T vehicle-loan dataset

Everything needed to go from a CSV to a working, explainable scoring product with an early-warning layer, and from there to a lender pilot

Working document for the project team · 25 October 2026

1. What we are building

We are building a **credit decision engine** that an NBFC or fintech could use to lend small-ticket, two-wheeler-type loans to self-employed and gig workers who have no CIBIL score, and to watch those loans for early trouble. It has three parts that share one code base.

Part	What it does	Who uses it	Built on
Score (Layer 1)	Takes an application and returns a 300–900 score, a decision band (approve / refer / decline) and three plain-language reasons	Credit officer at an NBFC; or the lender's loan-origination system through an API	L&T dataset, 2.33 lakh real Indian loans
Watch (Layer 2)	After disbursement, takes the borrower's recent cash-flow and repayment behaviour and returns a green / amber / red flag with the signals that triggered it	Collections and customer teams	Rule-based first version from the survey and L&T's first-EMI signal; trained version when lender data arrives
Portfolio	Aggregates scores and flags into approval rate, expected bad rate, PAR 30 / PAR 90 and compares with Indian benchmarks	Risk head, board, regulator	Model outputs + CRIF / MFIN / CIBIL figures

The difference between this and a normal academic project is that every step is built the way a lender would need it: an out-of-time test that mimics real deployment, calibrated probabilities that can be priced, reason codes a customer can be given, a fairness check a regulator can read, an API contract, a drift monitor, and a pilot design. The models themselves are standard; the discipline around them is what makes the thing usable.

> **The one fact that shapes everything**
>
> In the L&T data, `loan_default = 1` means **the borrower missed the first EMI on its due date**. This is not "defaulted some day"; it is early-payment default, the earliest possible signal of repayment stress. So Layer 1 on this data is really predicting "who will be in trouble within the first month", which is the same question Layer 2 asks later in the loan. That makes the two layers one story: predict first-instalment stress at origination, then keep watching for the same stress every week after.

2. The dataset, exactly

Source. L&T Financial Services released the data for the LTFS Data Science FinHack on Analytics Vidhya in 2019; it is mirrored on Kaggle (search "L&T Vehicle Loan Default Prediction"). Loans were disbursed in August–October 2018. `train.csv` has 233,154 rows and 41 columns with the label; `test.csv` has 112,392 rows without the label (use it only for checking feature pipelines, never for evaluation). Documented default rate is 21.7%. Licence: released for the hackathon; fine for academic and prototype use, cite L&T Financial Services and Analytics Vidhya.

2.1 Data dictionary

Column	Meaning	How we use it
UniqueID	Loan / customer identifier	Key only; never a feature
loan_default	1 if first EMI missed on due date	**Target**
disbursed_amount, asset_cost, ltv	Loan amount, vehicle price, loan-to-value	Loan burden; down-payment share = 1 − disbursed/asset_cost
branch_id, supplier_id, manufacturer_id, Current_pincode_ID, State_ID, Employee_code_ID	Branch, dealer, vehicle maker, pincode, state, logging employee	High-cardinality IDs → leakage-safe target encoding (Section 5); State_ID also for fairness check
Date.of.Birth, DisbursalDate	Two-digit-year dates (dd-mm-yy)	Age at disbursal; out-of-time split; disbursal month/weekday
Employment.Type	Salaried / Self employed (some blank)	Key segment: self-employed ≈ gig-type borrower
MobileNo_Avl_Flag, Aadhar_flag, PAN_flag, VoterID_flag, Driving_flag, Passport_flag	1 if the document was shared	KYC completeness index; Driving_flag is a rider signal
PERFORM_CNS.SCORE	Bureau score; 0 = no score	Bureau signal; **no_score flag defines the credit-invisible subgroup**
PERFORM_CNS.SCORE.DESCRIPTION	Text bucket, e.g. "No Bureau History Available", "Not Scored: Sufficient History Not Available", "A-Very Low Risk" … "M-Very High Risk"	Clean into ~6 groups; identifies why someone has no score
PRI.NO.OF.ACCTS, PRI.ACTIVE.ACCTS, PRI.OVERDUE.ACCTS, PRI.CURRENT.BALANCE, PRI.SANCTIONED.AMOUNT, PRI.DISBURSED.AMOUNT	Borrower's own existing loans at disbursal	Repayment history and leverage features
SEC.* (same six)	Loans where borrower is co-applicant / guarantor	Same, lower weight; mostly zero
PRIMARY.INSTAL.AMT, SEC.INSTAL.AMT	Existing EMI amounts	Existing obligation; new EMI estimate added
NEW.ACCTS.IN.LAST.SIX.MONTHS, DELINQUENT.ACCTS.IN.LAST.SIX.MONTHS	Recent new loans; recent delinquencies	Recent behaviour; strongest bureau-type signals
AVERAGE.ACCT.AGE, CREDIT.HISTORY.LENGTH	Strings like "1yrs 11mon"	Convert to months

Column	Meaning	How we use it
NO.OF_INQUIRIES	Loan enquiries	Credit-seeking intensity

2.2 Known quirks to handle on day one

- Dates are `dd-mm-yy`; a birth year "84" is 1984 but "01" is 2001. Parse with a century rule (year > 25 → 19xx) or you will get negative ages.
- `AVERAGE.ACCT.AGE` and `CREDIT.HISTORY.LENGTH` are text; convert "Xyrs Ymon" to months.
- `PERFORM_CNS.SCORE = 0` with description "No Bureau History Available" or "Not Scored: …" is **not** a bad score; it is the absence of one. Keep it as a flag and treat the score as missing, never as zero.
- `Employment.Type` has blanks (~3%); keep "Unknown" as its own category.
- `branch_id`, `supplier_id`, `Current_pincode_ID`, `Employee_code_ID` have hundreds to thousands of levels. Plain one-hot will explode and plain target encoding will leak. Use out-of-fold target encoding with smoothing, or drop `Employee_code_ID` entirely (it is an operational artefact, not a borrower trait).
- Some PRI/SEC amounts are negative or absurdly large; winsorise at the 1st and 99th percentiles.
- The class split is 78 / 22, so accuracy is meaningless; use AUC, KS and precision–recall.
- Expect modest AUC. The hackathon's best public scores were around 0.67–0.68. First-EMI default is noisy; a model at 0.66–0.68 out-of-time is a good result on this data, and the value is in the no-score subgroup and in the explainability, not in chasing 0.80.

3. Why this dataset fits the problem

Three things make L&T the right single dataset for a credit-invisible gig-worker project. First, **the borrowers**: two-wheeler loans to a population that is majority self-employed, with roughly half having no bureau score. A delivery rider financing a bike through an NBFC is exactly this row. Second, **the lender**: an Indian NBFC, which is the type of institution that originates most new-to-credit loans in India and the most likely adopter of the engine. Third, **the label**: first-EMI default is early stress, which lets Layer 1 and Layer 2 share one definition of "trouble". Its gap, no cash-flow history, is covered by the survey and the Account Aggregator sandbox (Section 8), and later by lender data.

Two caveats go in every presentation: the data is from 2018, so levels (default rate, ticket sizes) are dated even if relationships hold; and the borrowers are "gig-like", not confirmed gig workers. The Pune survey is what connects the model to actual riders.

4. Setup and data pipeline

4.1 Environment

```
python -m venv .venv && source .venv/bin/activate
pip install pandas numpy scikit-learn lightgbm shap lifelines optuna streamlit joblib
matplotlib category_encoders kaggle
kaggle datasets download -d mamtadhaker/lt-vehicle-loan-default-prediction -p data/raw --unzip
```

4.2 Repository layout

```
gigscore/
  data/raw/            train.csv, test.csv, data_dictionary.csv (git-ignored)
  data/processed/      features.parquet, splits.json
  src/load.py          read CSVs, parse dates, fix types
  src/features.py      build_features(df) -> feature frame
  src/encode.py        out-of-fold target encoding, WoE binning
  src/train.py         fit LR scorecard + LightGBM, save models/
  src/evaluate.py      AUC/Gini/KS/PSI/calibration, subgroup tables, plots
  src/explain.py       SHAP, reason codes
  src/fairness.py      subgroup metrics
  src/score.py         score(application_dict) -> {score, band, reasons}
  src/watch.py         early-warning rules on statement rows
  models/              lr_scorecard.pkl, lgbm.pkl, encoders.pkl, calibrator.pkl
  app/streamlit_app.py
  api/main.py          FastAPI wrapper around score() and watch()
  notebooks/           01_profile, 02_features, 03_train, 04_explain
  reports/             model_card.md, validation_report.md
```

4.3 Loading and cleaning (src/load.py)

```
import pandas as pd, numpy as np, re

def parse_date(s):
    d = pd.to_datetime(s, format="%d-%m-%y", errors="coerce")
    # two-digit years: anything after 2025 is really 19xx
    return d.where(d.dt.year <= 2025, d - pd.DateOffset(years=100))

def to_months(s):
    m = s.str.extract(r"(\d+)yrs\s*(\d+)mon").astype(float)
    return m[0] * 12 + m[1]

def load(path="data/raw/train.csv"):
    df = pd.read_csv(path)
    df["dob"] = parse_date(df["Date.of.Birth"])
    df["disb"] = parse_date(df["DisbursalDate"])
    df["age"] = ((df["disb"] - df["dob"]).dt.days / 365.25).round(1)
    df["acct_age_m"] = to_months(df["AVERAGE.ACCT.AGE"])
    df["hist_len_m"] = to_months(df["CREDIT.HISTORY.LENGTH"])
    df["Employment.Type"] = df["Employment.Type"].fillna("Unknown")
    df["no_score"] = (df["PERFORM_CNS.SCORE"] == 0).astype(int)
    df["bureau_score"] = df["PERFORM_CNS.SCORE"].replace(0, np.nan)
    for c in [c for c in df.columns if c.startswith(("PRI.", "SEC."))]:
        lo, hi = df[c].quantile([0.01, 0.99]); df[c] = df[c].clip(lo, hi)
    return df
```

4.4 The split that mimics deployment

A random split flatters every model. Lenders deploy a model trained on the past and use it on the future, so we do the same: train and validate on loans disbursed in August and September 2018, test on October 2018 (out-of-time). Inside the training window use 5-fold cross-validation for tuning. Report the random-split result too, but the headline number is out-of-time. Save the split indices to `splits.json` so every notebook uses the same rows.

5. Feature engineering

Features are grouped into the five families we have used throughout, which double as the CAMEL-style mapping. Every feature below can be computed from the dictionary columns; nothing here needs data we do not have.

Family	Feature	Formula / definition	Why it matters for a gig borrower
Capacity (C)	ltv	As given	Higher LTV = less own money in the bike = higher risk
Capacity	down_payment_share	1 − disbursed_amount / asset_cost	Skin in the game
Capacity	new_emi_est	disbursed_amount × r(1+r)^n / ((1+r)^n − 1), r = 1.5%/month, n = 36 (document the assumption)	Estimated new EMI
Capacity	total_emi	PRIMARY.INSTAL.AMT + SEC.INSTAL.AMT + new_emi_est	Obligation stack
Capacity	leverage	PRI.CURRENT.BALANCE / (PRI.SANCTIONED.AMOUNT + 1)	How much of existing credit is still owed
Asset quality (A)	no_score	PERFORM_CNS.SCORE == 0	Defines the credit-invisible subgroup
Asset quality	bureau_score	Score if > 0 else missing	Bureau signal where it exists
Asset quality	score_desc_grp	Collapse description to: NoHistory / NotScored / LowRisk (A–D) / Medium (E–H) / High (I–M)	Why a score is missing matters
Asset quality	overdue_ratio	PRI.OVERDUE.ACCTS / (PRI.NO.OF.ACCTS + 1)	Share of accounts in trouble
Asset quality	delinq_6m	DELINQUENT.ACCTS.IN.LAST.SIX.MONTHS	Strongest single bureau-type signal
Asset quality	active_ratio	PRI.ACTIVE.ACCTS / (PRI.NO.OF.ACCTS + 1)	Still-running vs closed loans
Management (M)	kyc_count	Sum of the six document flags	Documentation discipline
Management	has_pan, has_driving	Flags as given	PAN ≈ formal footprint; driving licence ≈ rider
Management	inquiries	NO.OF_INQUIRIES	Credit-hungry behaviour
Management	new_accts_6m	NEW.ACCTS.IN.LAST.SIX.MONTHS	Recent stacking of loans
Earnings (E)	employment_self	Employment.Type == Self employed	Core segment flag
Earnings	age, age_band	From DOB	Life-stage; also fairness attribute
Earnings	asset_cost_log	log(asset_cost)	Proxy for earning aspiration / segment
Liquidity (L)	hist_len_m, acct_age_m	Converted strings	Thin vs thick file
Liquidity	first_loan	PRI.NO.OF.ACCTS == 0	True first-time borrower
Context	state_te, branch_te, supplier_te, manufacturer_te	Out-of-fold target encoding with smoothing (m = 50)	Local risk; dealer quality
Context	disb_month, disb_weekday	From DisbursalDate	Seasonality / month-end push

5.1 Leakage-safe target encoding (src/encode.py)

```
from sklearn.model_selection import KFold
def oof_target_encode(df, col, y, m=50, folds=5, seed=42):
    prior = y.mean(); out = pd.Series(np.nan, index=df.index)
    for tr, va in KFold(folds, shuffle=True, random_state=seed).split(df):
        stats = y.iloc[tr].groupby(df[col].iloc[tr]).agg(["sum", "count"])
        enc = (stats["sum"] + prior * m) / (stats["count"] + m)
        out.iloc[va] = df[col].iloc[va].map(enc).fillna(prior).values
    full = y.groupby(df[col]).agg(["sum", "count"])
    mapping = ((full["sum"] + prior * m) / (full["count"] + m)).to_dict()
    return out, mapping   # use `mapping` for the test set and in production
```

Feature screening: drop anything with information value below 0.02 for the scorecard, keep the more interpretable of any pair correlated above 0.8, and flag features with PSI above 0.25 between August–September and October. Expect `delinq_6m`, `overdue_ratio`, `ltv`, `score_desc_grp`, `kyc_count`, `employment_self`, `age` and the encoded `branch`/`supplier` to carry most of the signal.

6. Modelling

6.1 Baseline: weight-of-evidence logistic scorecard

Bin each feature into 5–10 groups (quantile bins for numerics, categories for the rest), replace values with weight of evidence, fit L2-regularised logistic regression, then scale to points: with a base score of 600 at 20:1 odds and 20 points to double the odds, factor = 20 / ln(2) and offset = 600 − factor × ln(20). The result is a points table a credit officer can read and a regulator can audit. This is the model you would actually deploy first; LightGBM is the challenger that tells you how much you are leaving on the table.

6.2 Challenger: LightGBM

```
import lightgbm as lgb
params = dict(objective="binary", learning_rate=0.03, num_leaves=31, min_child_samples=200,
              feature_fraction=0.8, bagging_fraction=0.8, bagging_freq=1, lambda_l2=5.0,
              scale_pos_weight=3.5, metric="auc", verbose=-1, seed=42)
mono = {"ltv": 1, "delinq_6m": 1, "overdue_ratio": 1, "inquiries": 1, "kyc_count": -1,
"down_payment_share": -1}
params["monotone_constraints"] = [mono.get(f, 0) for f in features]
model = lgb.train(params, lgb.Dataset(X_tr, y_tr), 3000, valid_sets=[lgb.Dataset(X_va, y_va)],
                  callbacks=[lgb.early_stopping(100), lgb.log_evaluation(200)])
```

Monotone constraints are not decoration: they guarantee that more delinquencies can never lower risk and more documents can never raise it, which is what makes the model defensible to a risk committee. Tune `num_leaves`, `min_child_samples`, `feature_fraction` and `lambda_l2` with Optuna on the validation fold (50 trials is enough). Then **calibrate** the probabilities with isotonic regression on the validation set so that "12% predicted" means 12% observed; calibrated probabilities are what pricing and provisioning need.

6.3 The three comparisons that answer the research question

- **Bureau-only vs alternative-only vs combined.** Bureau-only = bureau_score, score_desc_grp, PRI/SEC aggregates, delinquencies, inquiries, history length. Alternative-only = ltv, down-payment share, KYC flags, employment, age, asset cost, state/branch/supplier encodings, disbursal timing. Combined = all. Fit all three with both models.
- **No-score subgroup.** Evaluate every model on rows with `no_score = 1`. Here bureau-only collapses to almost nothing, and the gain from alternative features is your headline: "for borrowers with no CIBIL score, alternative data lifts AUC from X to Y".
- **Self-employed subgroup.** Same evaluation on `Employment.Type = Self employed`, the gig-like segment.

6.4 Decision bands

Do not use a 0.5 cut-off. Choose the cut-offs from the approval-rate versus bad-rate curve on the test set: for example, approve the safest 60% (expected first-EMI default below the portfolio's current rate), refer the next 20% for manual review, decline the rest. Show the lender the curve and let them pick; the engine stores the chosen thresholds in a config file, not in code.

7. Evaluation, explainability and fairness

What	How	Target on this data
Discrimination	AUC, Gini = 2·AUC − 1, KS; out-of-time October test; bootstrap 95% CI	AUC 0.66–0.68 overall; gain of ≥ 0.03 on the no-score subgroup from alternative features
Calibration	Reliability curve by decile; Brier score; before and after isotonic	Predicted within ±2 pp of observed per decile
Stability	PSI of each feature and of the score between Aug–Sep and Oct	PSI < 0.10 stable; > 0.25 investigate
Business	Approval-rate vs bad-rate curve; expected loss at chosen cut-off vs "approve all" and vs "bureau-only"	Quantify extra approvals at the same bad rate
Explainability	SHAP summary and dependence plots; top-3 SHAP contributions per applicant → reason codes	Every decision has three readable reasons
Fairness	Approval rate, TPR and FPR by age band and State_ID (no gender in data); feature–attribute correlations	Gaps > 5 pp flagged; mitigation tested

7.1 Reason codes

Map each feature to a sentence so SHAP output becomes language. For example: `ltv` → "The loan is a high share of the vehicle's price"; `delinq_6m` → "A loan was overdue in the last six months"; `kyc_count` → "Fewer identity documents were provided"; `down_payment_share` → "A larger down payment was made" (positive). The app shows the three strongest negatives for a decline and the three strongest positives for an approval. This is the format RBI's FREE-AI principles and standard adverse-action practice expect.

7.2 Model card and validation report

Write `reports/model_card.md` (purpose, data, features, training window, metrics with CIs, subgroup results, limitations, intended and non-intended use) and `reports/validation_report.md` (tests run, PSI, calibration, fairness, sign-off). These two files are what RBI's 2026 draft model-risk guidance asks a lender to have, and they are what turns a notebook into something a risk committee can approve.

8. Layer 2: the early-warning layer with this data

L&T has no cash-flow history, so Layer 2 cannot be trained on it. We build it honestly in three steps, each useful on its own.

Step 1 — the first-instalment signal is already there. Because the L&T label is first-EMI default, the Layer 1 score is itself the earliest warning: a borrower approved in the "refer" band gets a weekly check from day one. Store the origination score with the loan; it is the prior that Layer 2 updates.

Step 2 — a rule-based monitor built from the survey and the literature. The Pune survey tells you which signals precede a missed EMI (falling earnings, no buffer, late bills, high EMI-to-income). Encode them as transparent rules on a borrower's last 8–12 weeks of bank/UPI statement rows:

Signal (from statement rows)	Amber if	Red if
Inflow trend	Last 4-week mean inflow < 80% of 12-week mean	< 60%
Inflow gaps	1 week with zero inflow in last 8	2+ weeks
Balance before EMI date	Minimum balance in the 5 days before due date < 1.0 × EMI	< 0.5 × EMI
Utility / rent standing orders	One late or bounced in last 3 months	Two or more
Obligation ratio	Total EMIs > 35% of 4-week net inflow	> 50%
Cash-out speed	> 70% of each inflow withdrawn within 48 h	> 90%
Combination	Any two amber	Any red, or three amber

Rules are not a weakness here: a transparent rule set is what most NBFC collections teams actually run, and it is auditable. Validate the rules on the consented rider statements you collect (Section 10.3): the riders who reported missed EMIs should trip more flags.

Step 3 — the trained version, when data arrives. The `watch.py` module is written so that the rule features are the same columns a Cox survival model or LightGBM discrete-time model would consume. When a lender shares loan-week data, or if you add Home Credit's instalment table as a stand-in, you fit the model on the same frame and replace the rules with hazard-based thresholds without touching the app.

9. The product: from script to app to API

9.1 Architecture

Stage	Stack	What it proves
MVP (weeks 1–3)	Streamlit app loading `models/*.pkl`; CSV upload for statements; runs locally and on Streamlit Community Cloud or Hugging Face Spaces	End-to-end demo anyone can open from a link
Prototype (weeks 4–5)	FastAPI backend exposing `/score` and `/watch`; Streamlit or a simple React front end calling it; SQLite/Postgres for applications and flags; Account Aggregator sandbox (Setu / Finvu developer sandbox) for the "connect bank" flow with test accounts	Same models behind an API a lender's system could call; realistic consent flow
Pilot-ready (later)	Docker; model registry (MLflow); PSI drift monitor; audit log of every decision and reason; role-based access; data stored in India	What an NBFC's IT and compliance teams will ask for

9.2 Screens

- **Apply.** Form with the fields a loan officer has (loan amount, vehicle price, age, employment type, state, documents shared, existing loans and EMIs, bureau score if any). Output: score, band, three reasons, and the approval-rate curve showing where this applicant sits.
- **Connect & Watch.** Borrower selects their bank (sandbox), consents, statements load; or the user uploads a UPI/bank CSV. Output: 12-week inflow chart, flag colour, triggered rules, suggested action (reminder call, EMI-date shift, restructuring offer).
- **Portfolio.** Table of all scored applications and watched loans; approval rate, expected bad rate, flags by colour, PAR 30 / PAR 90 of the book (simulated and labelled as such until real), against CRIF MicroLend and CIBIL figures.
- **About the model.** Model card, validation summary, fairness table, data provenance. Lenders and examiners open this page first.

9.3 API contract (api/main.py)

```
POST /score
{ "disbursed_amount": 65000, "asset_cost": 82000, "age": 27, "employment_type": "Self
employed",
  "state_id": 6, "docs": {"aadhar":1,"pan":0,"voterid":1,"driving":1,"passport":0,"mobile":1},
  "bureau_score": null, "pri_no_of_accts": 0, "pri_overdue_accts": 0, "primary_instal_amt": 0,
  "new_accts_6m": 0, "delinq_6m": 0, "inquiries": 1, "credit_history_months": 0 }
→ { "score": 612, "pd": 0.19, "band": "refer",
    "reasons": ["High loan-to-value", "No credit history available", "PAN not provided"],
    "model_version": "lgbm-2026.10", "explain_id": "…" }

POST /watch
{ "loan_id": "…", "emi": 3300, "due_day": 5, "rows": [{"date":"2026-09-
01","credit":1450,"debit":900,"balance":2200}, …] }
→ { "flag": "amber", "triggers": ["inflow_trend_80", "obligation_35"], "lead_estimate_days":
18,
    "suggested_action": "reminder_call" }
```

9.4 Non-negotiables for a lending product

- Consent text shown before any data is pulled; no device scraping, contacts or call logs (RBI Digital Lending Directions 2025).

- Every decision logged with inputs, model version, score, band and reasons; retrievable for a customer complaint.
- Protected attributes never enter the model; they are stored only for the fairness report.
- A kill switch: the app can fall back to the logistic scorecard or to manual review if the LightGBM model drifts (PSI > 0.25) or errors.
- Data stays in India; sandbox and demo data are synthetic and labelled.

10. From prototype to real: the pilot plan

10.1 What a lender needs to see

A risk head will ask four questions: does it rank risk better than what we use today, is it stable, can we explain a decline to a customer, and what happens to our bad rate if we approve more. Sections 6–7 produce exactly those answers. Package them as a ten-slide deck plus the app link and the model card.

10.2 Pilot design

- **Champion–challenger, shadow mode.** The NBFC keeps its current process; our engine scores the same applications in parallel for 8–12 weeks without affecting decisions. Compare first-EMI and 30-DPD outcomes by score band.
- **Retrain on their book.** The pipeline retrains on the lender's historical loans with one config change; the Streamlit/API layer does not change.
- **Success metrics.** Gini on their data; approvals added at the same bad rate; share of declines with reasons the customer accepted; flags that preceded actual delinquency; false-alarm rate.
- **Who to approach.** NBFCs and fintechs already lending to riders or financing two-wheelers for self-employed buyers; bike-finance arms of large NBFCs; platform-linked lenders. Go through the faculty guide, alumni and LinkedIn; lead with the app link.

10.3 The small real-data step you can do now

Ask 30–50 of the riders you survey for consent to share their last six months of UPI or bank statement exports and whether they have an EMI and have missed one. Run `watch.py` on those statements. If the riders who reported a missed EMI trip more flags than those who did not, you have a validation of Layer 2 on real Pune gig workers that no public dataset can give. It is small; it is also the most convincing slide in the deck.

11. Work plan for the remaining weeks

Week	Build	Output
1	Download; profiling notebook; load.py with date and string fixes; out-of-time split; feature table v1	features.parquet; profiling summary with no-score and self-employed shares confirmed
2	WoE scorecard; LightGBM with monotone constraints; Optuna; isotonic calibration; three feature-set comparison; subgroup results	Results table with CIs; saved models
3	SHAP; reason-code map; fairness by age band and state; PSI; model card and validation report; Streamlit Apply screen	App v1 live on a public link
4	watch.py rules; Connect & Watch screen with CSV upload and AA sandbox; Portfolio screen; survey cross-tabs feeding rule thresholds	App v2; rules validated on survey
5	FastAPI wrapper; decision logging; kill switch; README; ten-slide lender deck; rider-statement check if collected	Prototype + deck
6	Report writing; rehearsal; outreach e-mails to two or three lenders	Submission; pilot conversations started

Roles that work for four or five people: one owns data and features, one owns models and evaluation, one owns explainability, fairness and the model card, one owns the app and API, and the lead owns the survey link, the deck and lender outreach. Everyone commits to the same repository from week 1.

12. Honest limits, and why the solution is still real

- The training data is from 2018 and from one lender; levels will differ from 2026, relationships are more durable. The pipeline retrains on any lender's book.
- AUC will be modest because first-EMI default is noisy. The product value is extra approvals at the same risk for people who are currently declined by default, plus explainability, not a high AUC.
- Borrowers are gig-like, not confirmed gig workers. The Pune survey and rider statements are the bridge; the pilot is the proof.
- Layer 2 starts as rules. That is how most lenders run early warning today; the trained version is a config change away once data exists.

What makes this an actual solution rather than an assignment is that at the end you will have a scored, explained, calibrated decision engine on real Indian NBFC loans, a monitoring layer whose rules come from real Pune riders, an app a lender can open, and a pilot plan with the numbers a risk head needs. That is the complete package a fintech would build before its first customer, and it is achievable in six weeks on one dataset.

Appendix A: starter training script (src/train.py)

```
import pandas as pd, numpy as np, joblib, json, lightgbm as lgb
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score
from sklearn.isotonic import IsotonicRegression
from load import load; from features import build_features; from encode import
oof_target_encode

df = load(); df = build_features(df)
oot = df["disb"] >= "2018-10-01"                        # out-of-time test = October
tr, te = df[~oot].copy(), df[oot].copy(); y_tr, y_te = tr.loan_default, te.loan_default
for col in ["State_ID", "branch_id", "supplier_id", "manufacturer_id"]:
    tr[col+"_te"], mp = oof_target_encode(tr, col, y_tr); te[col+"_te"] =
te[col].map(mp).fillna(y_tr.mean())
feats = json.load(open("data/processed/feature_list.json"))
va = tr.sample(frac=0.2, random_state=42); fit = tr.drop(va.index)
m = lgb.train(params, lgb.Dataset(fit[feats], fit.loan_default), 3000,
              valid_sets=[lgb.Dataset(va[feats], va.loan_default)],
callbacks=[lgb.early_stopping(100)])
cal = IsotonicRegression(out_of_bounds="clip").fit(m.predict(va[feats]), va.loan_default)
p = cal.predict(m.predict(te[feats]))
print("OOT AUC all:", roc_auc_score(y_te, p))
ns = te.no_score == 1; print("OOT AUC no-score:", roc_auc_score(y_te[ns], p[ns]))
joblib.dump({"model": m, "cal": cal, "feats": feats}, "models/lgbm.pkl")
```

Appendix B: scoring function used by the app and API (src/score.py)

```
import joblib, numpy as np, shap
B = joblib.load("models/lgbm.pkl"); explainer = shap.TreeExplainer(B["model"])
REASON = {"ltv": "The loan is a high share of the vehicle price", "delinq_6m": "A loan was
overdue in the last six months",
          "kyc_count": "Fewer identity documents were provided", "no_score": "No credit-bureau
history is available",
          "down_payment_share": "A larger down payment was made", "overdue_ratio": "Existing
loans are overdue", "age": "Applicant age"}
def to_points(pd_):  # 600 at 20:1 odds, 20 points to double the odds
    odds = (1 - pd_) / pd_; return int(round(600 + (20 / np.log(2)) * (np.log(odds) -
np.log(20))))
def score(x: dict, cfg):
    row = featurize(x)                                    # same build_features logic for one
row
    pd_ = float(B["cal"].predict(B["model"].predict(row[B["feats"]]))[0])
    s = to_points(pd_); band = "approve" if s >= cfg["approve"] else "refer" if s >=
cfg["refer"] else "decline"
    sv = explainer.shap_values(row[B["feats"]])[0]
    order = np.argsort(sv)[::-1] if band == "decline" else np.argsort(sv)
    reasons = [REASON.get(B["feats"][i], B["feats"][i]) for i in order[:3]]
    return {"score": s, "pd": round(pd_, 3), "band": band, "reasons": reasons}
```